Compare Hash Values Using Linux Tools

Daniel Dela Dzikunu | June 11, 2026

Project Description

Hash functions are a cornerstone of data integrity verification in cybersecurity. A hash function is a one-way algorithm that takes any input and produces a fixed-length digest — also called a hash value — that uniquely represents the input's contents. Even a single-bit difference between two files will produce entirely different hash values, making hashing an ideal mechanism for detecting tampering, verifying file authenticity, and identifying malicious programs masquerading as legitimate ones.

This lab simulates a scenario in which two files appear visually identical when printed to the terminal, yet are actually different at the binary level. The objective was to use Linux command-line tools to generate SHA-256 hash values for both files, redirect those values to new files, and then perform a byte-level comparison to confirm the discrepancy. The tools exercised were **sha256sum**, **cat**, and **cmp**.

Task 1 — Generate Hashes for Files

The lab environment began with two files, **file1.txt** and **file2.txt**, located in the analyst home directory. Both files contained the EICAR Standard Antivirus Test string — a well-known benign sequence used to test antivirus software without using actual malware. Running **ls** confirmed the presence of both files, and **cat** printed their contents to the terminal. The output appeared identical in both cases, illustrating why visual inspection alone is insufficient for integrity verification.

```
ls
cat file1.txt
cat file2.txt
```

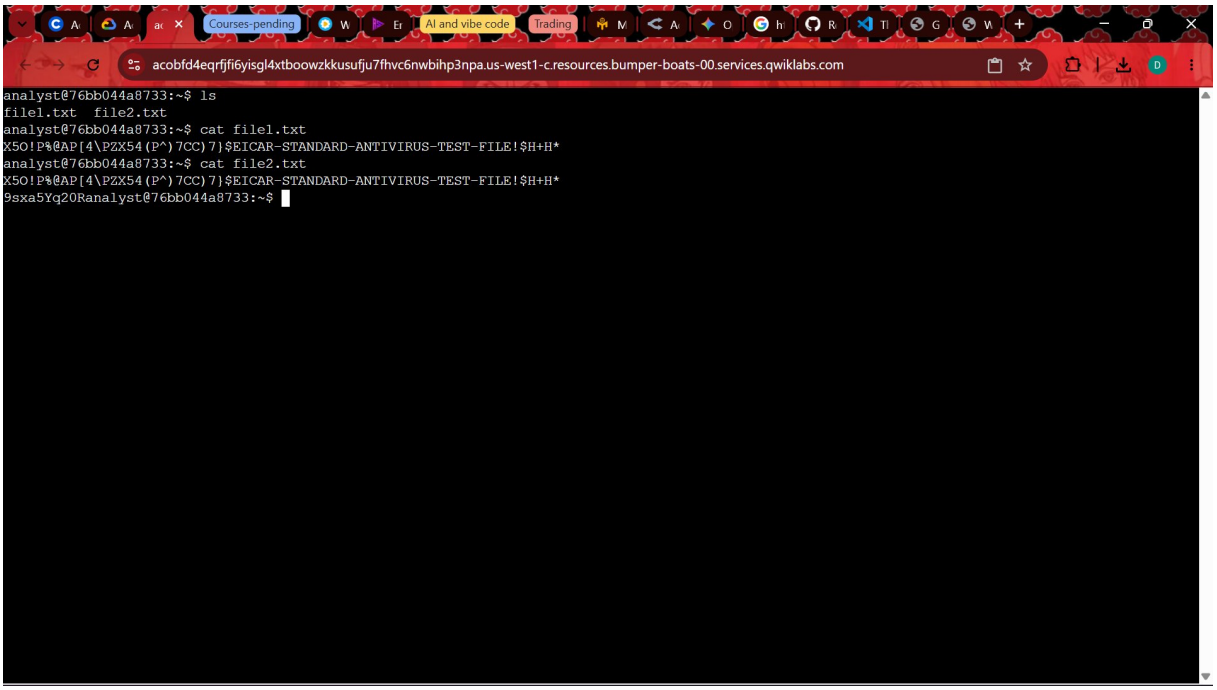

Figure 1 — ls confirms both files exist; cat reveals visually identical EICAR content

To look past the surface, **sha256sum** was run against each file. The SHA-256 algorithm generates a 256-bit (64-character hexadecimal) digest. Despite the contents appearing the same, the two digests were completely different — confirming that the files differ at a level invisible to **cat**. This demonstrates the avalanche effect inherent to cryptographic hash functions: even a single hidden character or byte difference cascades into a wholly distinct hash value.

```
sha256sum file1.txt
# → 131f95c51cc819465fa1797f6ccacf9d494aaaff46fa3eac73ae63ffbdfd8267  file1.txt

sha256sum file2.txt
# → 2558ba9a4cad1e69804ce03aa2a029526179a91a5e38cb723320e83af9ca017b  file2.txt
```

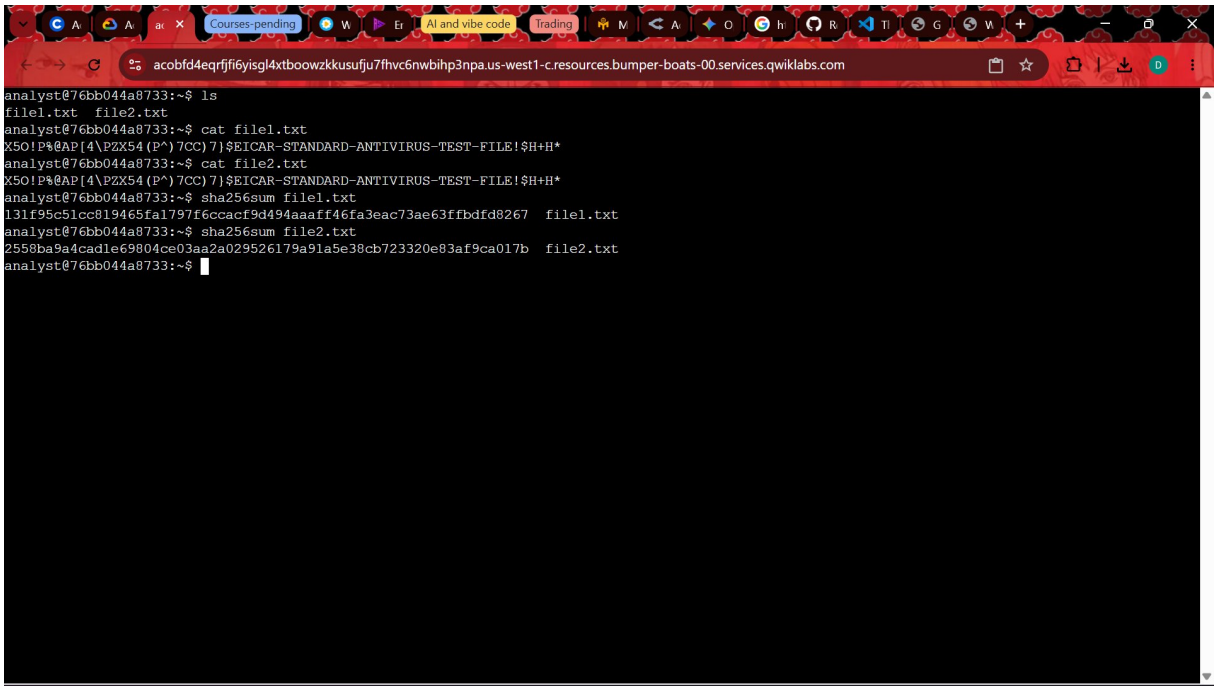

Figure 2 — sha256sum produces distinct 64-character digests for each file

Task 2 — Compare Hashes

With the hash values confirmed to differ, the next step was to persist them for comparison. The **>>** redirection operator appended each **sha256sum** output to a dedicated file — **file1hash** and **file2hash** respectively. Redirecting to files rather than reading directly from the terminal is a practical pattern in SOC workflows, where hash values may need to be logged, shared with other analysts, or fed into automated comparison pipelines.

```
sha256sum file1.txt >> file1hash
sha256sum file2.txt >> file2hash
```

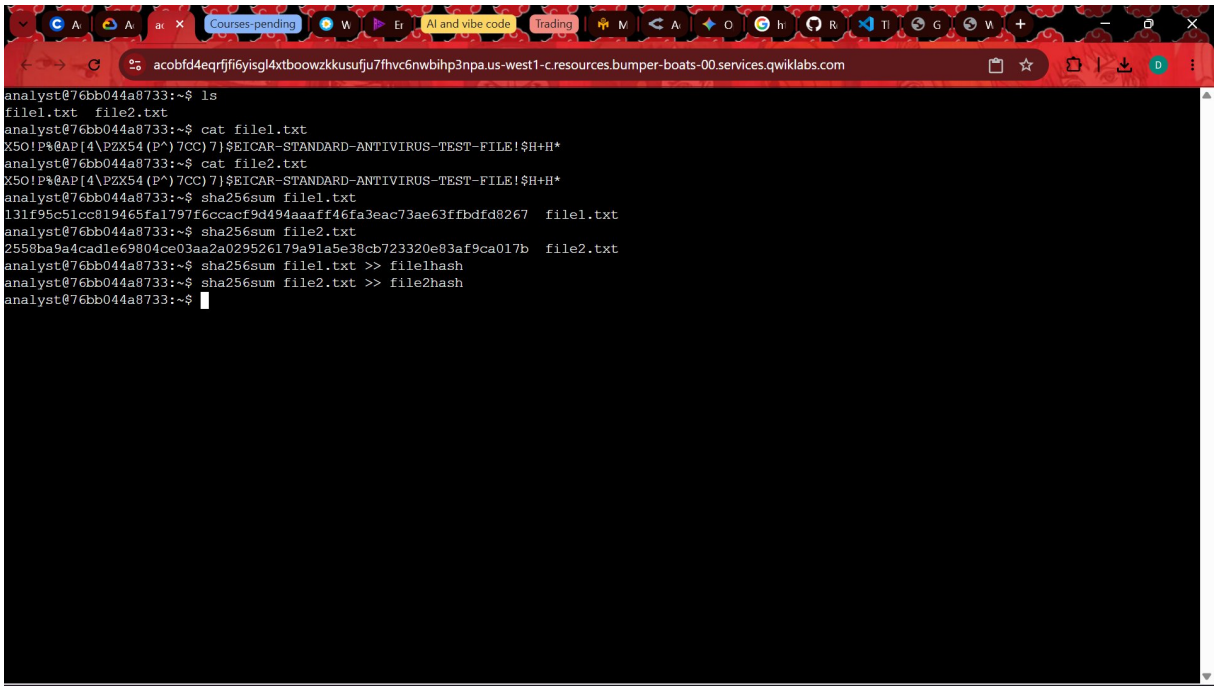

Figure 3 — Hash values redirected to file1hash and file2hash using the >> operator

Finally, the **cmp** command performed a byte-by-byte comparison of the two hash files. Unlike **diff**, which compares line by line, **cmp** operates at the character level and reports the exact position of the first divergence. The output confirmed a mismatch at character 1 of line 1 — the very first byte — meaning the two hash strings differ from the outset, as expected given the completely different digests. This level of precision is particularly useful when analysts need to verify whether a suspected file modification occurred and, if so, at what point in the data.

```
cmp file1hash file2hash
# → file1hash file2hash differ: char 1, line 1
```

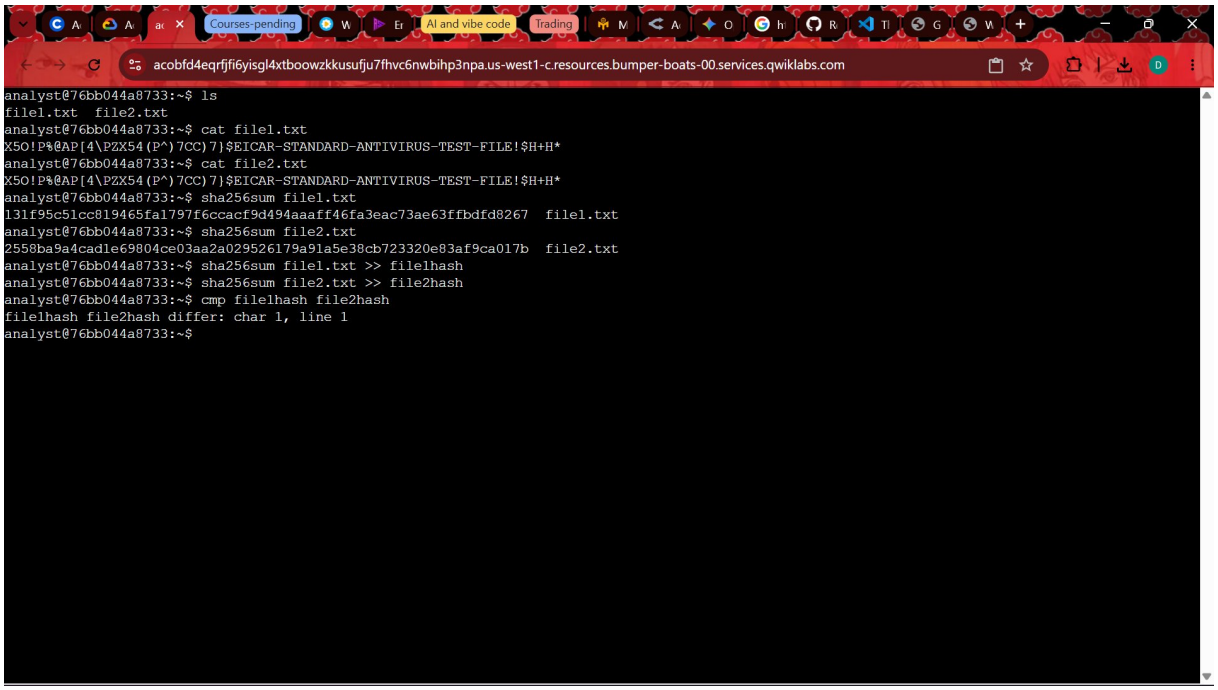

Figure 4 — cmp reports the first byte mismatch between the two hash files

Summary

This lab reinforced a fundamental data integrity technique: using cryptographic hashing to detect file tampering that is invisible to plain-text inspection. The three tools covered — **sha256sum**, **cat**, and **cmp** — form a lightweight but powerful verification toolkit available on virtually every Linux system. In a real-world SOC context, this workflow maps directly to validating software downloads against published checksums, detecting unauthorized changes to configuration files, or identifying malware that has replaced a legitimate binary while keeping the same filename and apparent content.